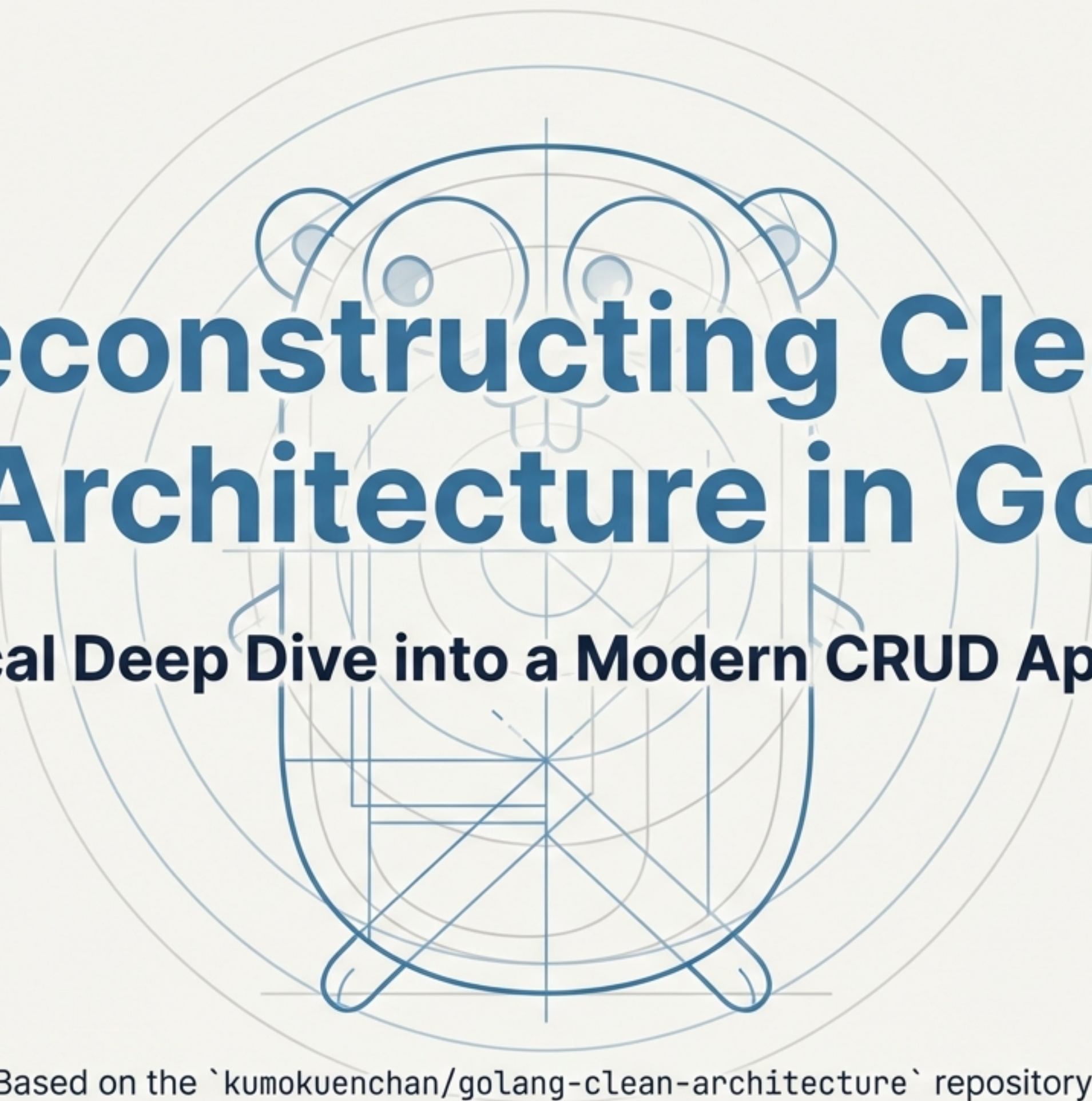




Deconstructing Clean Architecture in Go

A Practical Deep Dive into a Modern CRUD Application

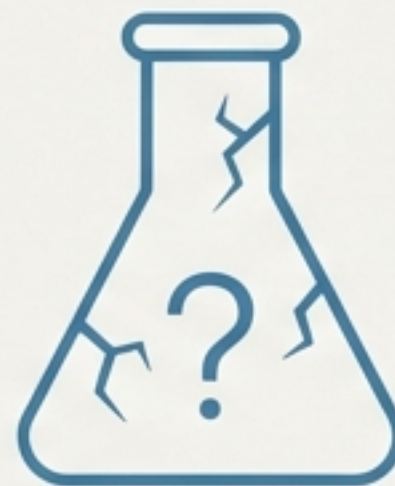
Based on the ``kumokuenchan/golang-clean-architecture`` repository.

Why do good projects become hard to maintain?



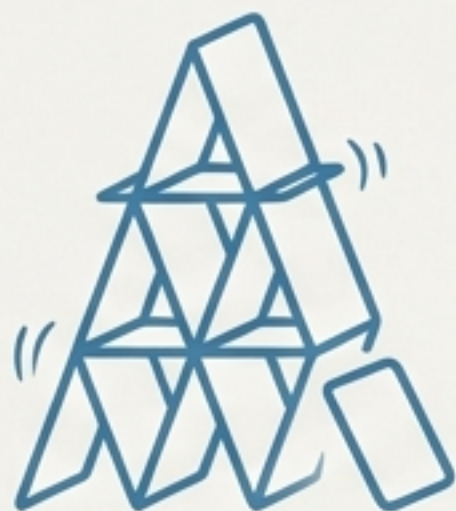
Framework Lock-in

Business logic becomes entangled with a specific web framework, making it difficult to evolve.



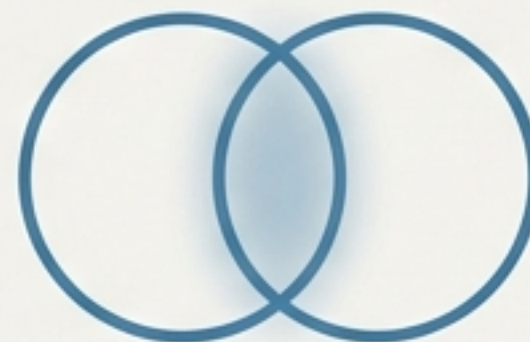
Poor Testability

Core business rules cannot be tested without spinning up a database, a web server, or the UI.



Brittle & Fragile

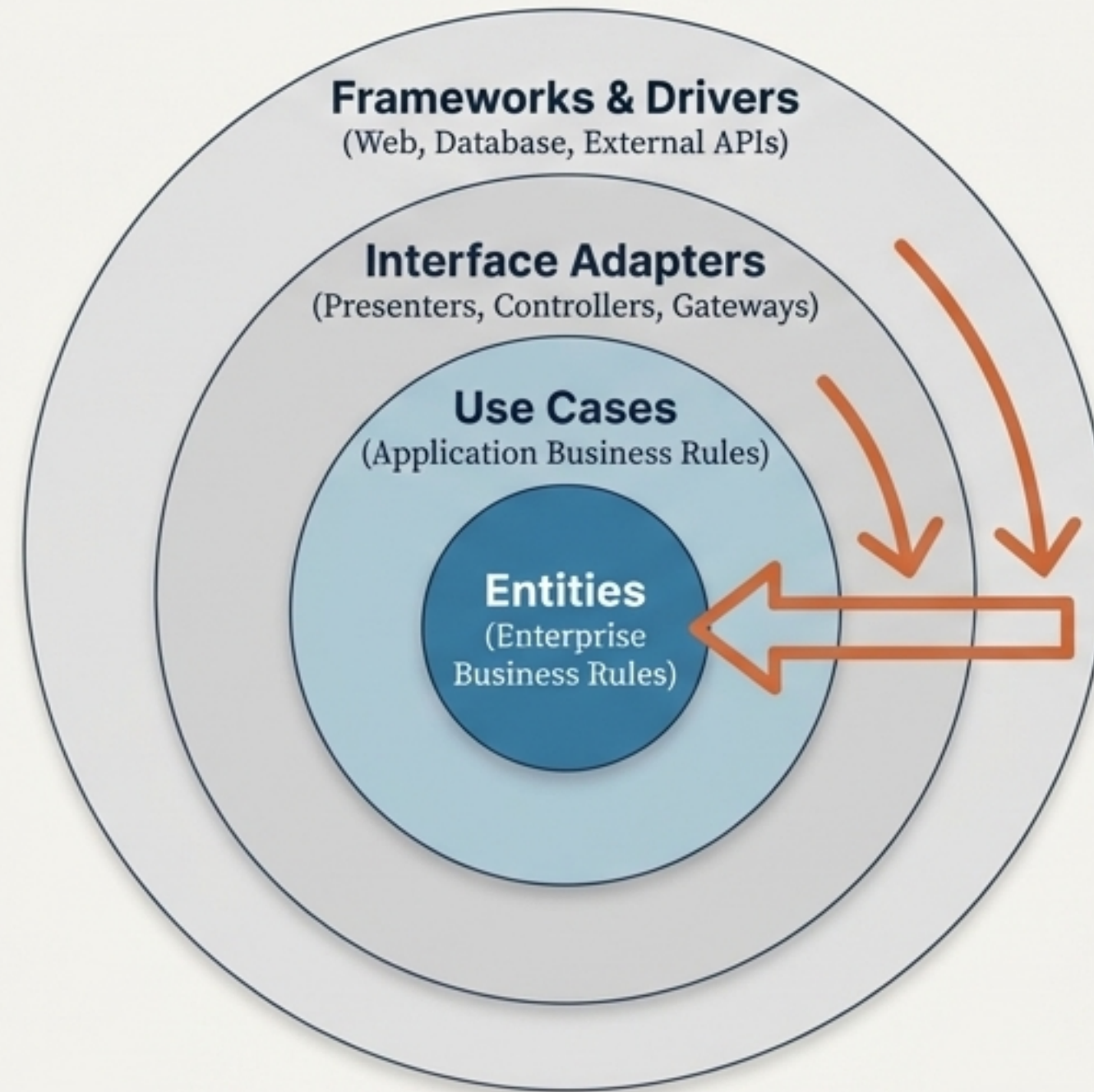
A small change in the database schema can cause cascading failures all the way up to the API responses.



Blurred Boundaries

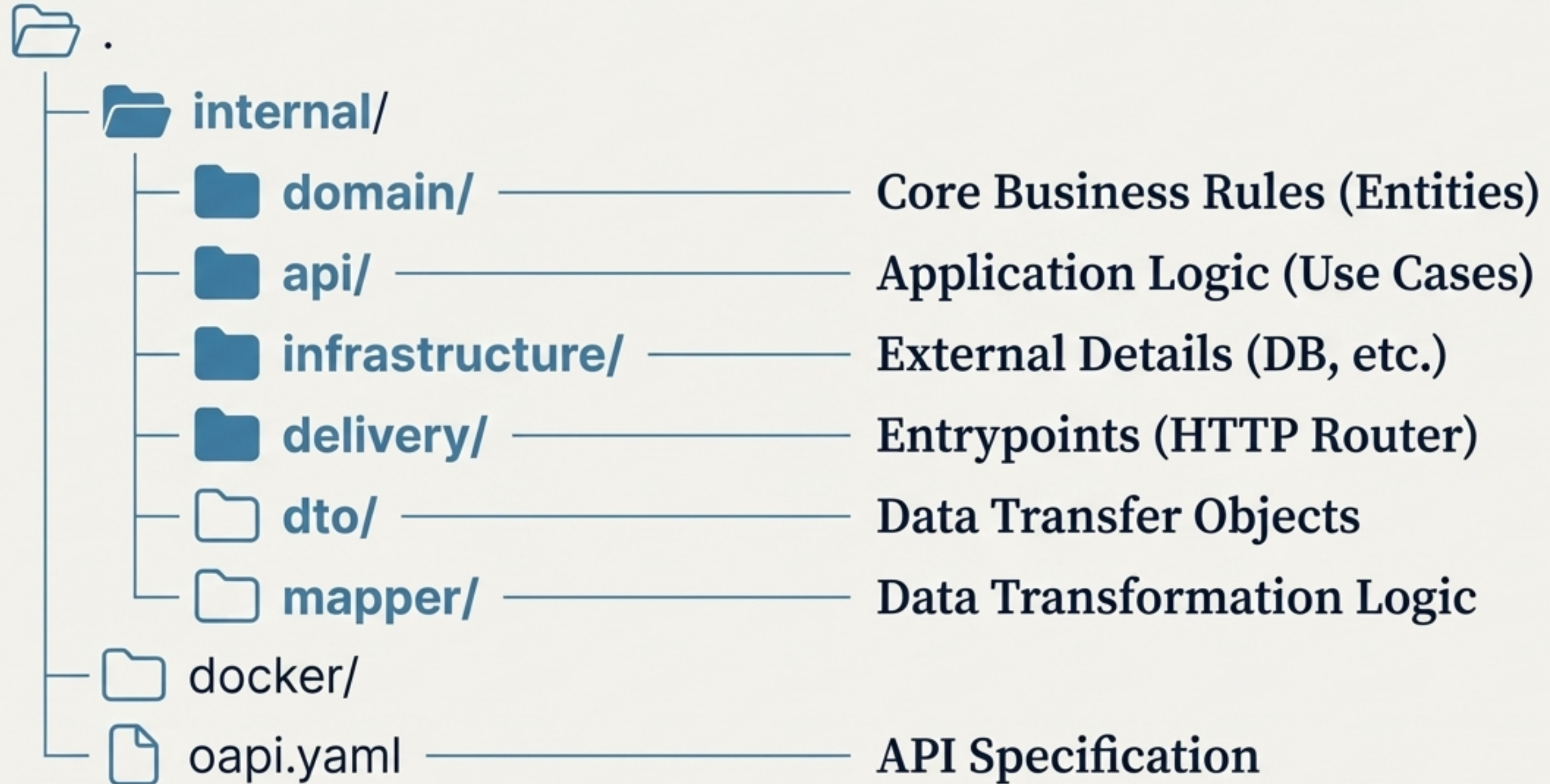
No clear separation of concerns, making it difficult to know where new code should live or what a specific module is responsible for.

A Blueprint for Separation of Concerns

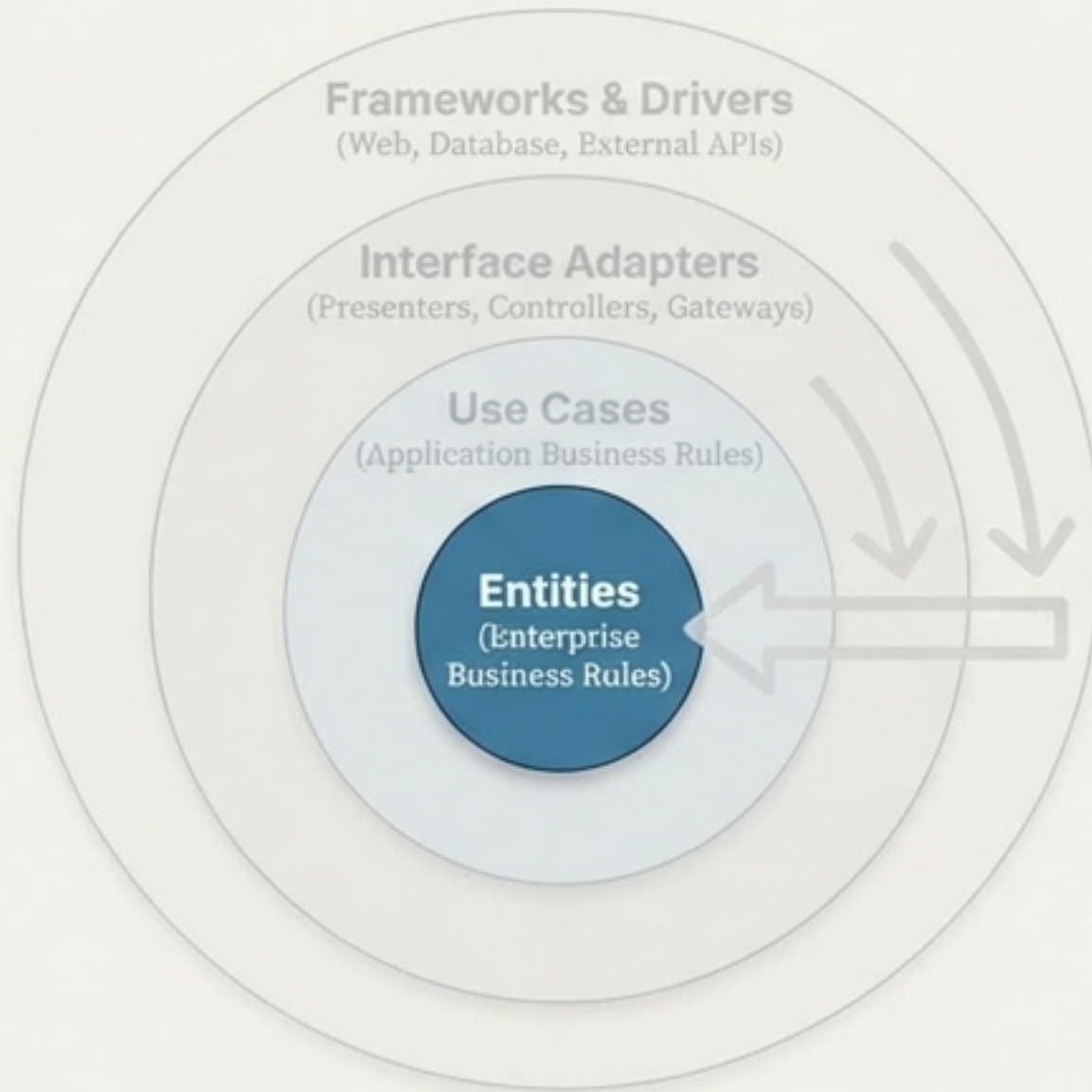


The Dependency Rule: Source code dependencies can **only** point inwards.

From Blueprint to Reality: Project Structure



Layer 1: The Domain – The Heart of the Business



- **Purpose:** Contains the core business entities and rules.
- **Dependencies:** None. This is the most independent layer.

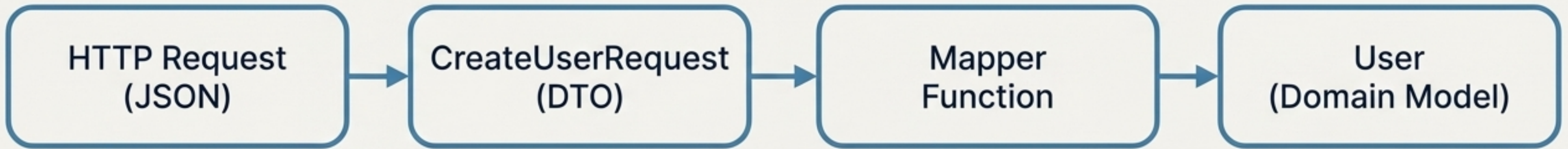
```
// The core business entity
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
    // ...
}
```

```
// Defines the contract for data access, not the implementation
type UserRepository interface {
    Create(user *User) error
    GetByID(id int) (*User, error)
    GetAll() ([]*User, error)
    // ...
}
```

Defines the *contract*, not the implementation. This is key to dependency inversion.

Crossing Boundaries: Data Transfer Objects & Mappers

To decouple internal domain models from external API contracts. Prevents the domain from being polluted with presentation or transport-layer concerns like JSON tags or validation.

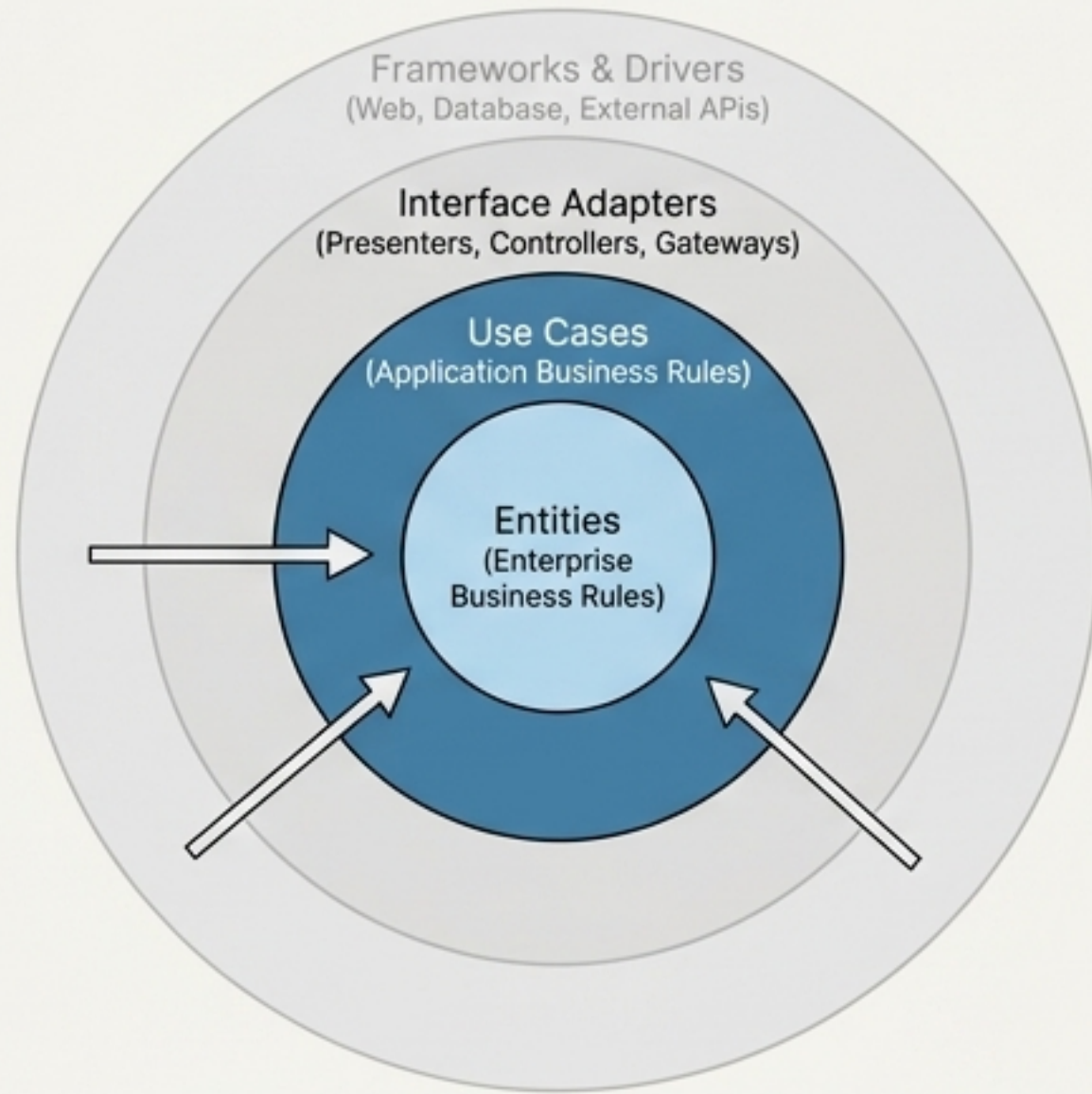


```
// DTO for incoming requests
type CreateUserRequest struct {
    Name string `json:"name" example:"John Doe"`
    Email string `json:"email" example:"john@example.com"`
}
```

```
// Mapper function using jinzhu/copier
func MapCreateRequestToDomain(req *dto.CreateUserRequest) *domain.User {
    user := &domain.User{}
    copier.Copy(user, req) // Automatically copies fields
    user.CreatedAt = time.Now()
    user.UpdatedAt = time.Now()
    return user
}
```

Reduces boilerplate and errors via automated object mapping.

Layer 2: The API – Orchestrating Business Rules

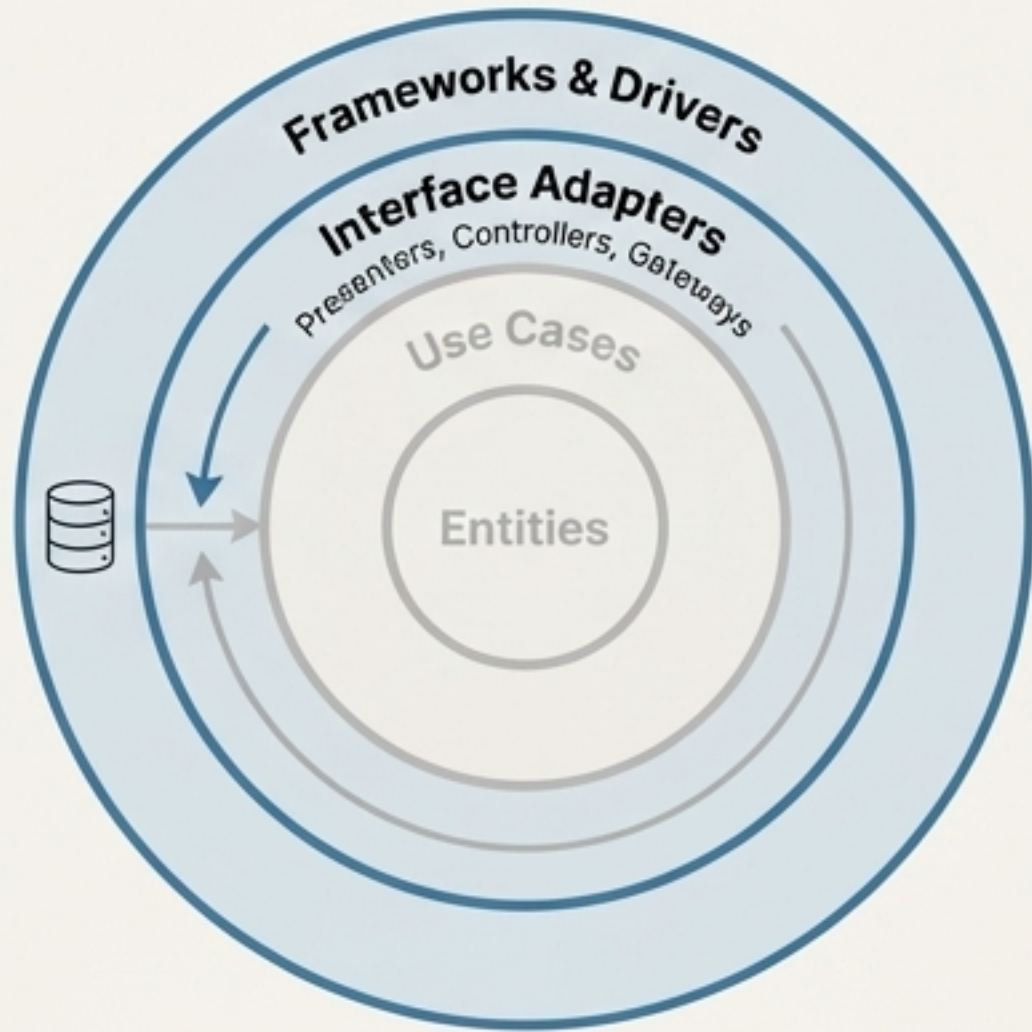


- ****Purpose****: Implements specific use cases (e.g., “Create User”) and orchestrates the flow of data.
- ****Dependencies****: Depends inward on the Domain (interfaces), DTOs, Mappers, and Presenters.

```
func (s *Server) CreateUser(w http.ResponseWriter, r *http.Request) {  
    // 1. Decode request into a DTO  
    var req dto.CreateUserRequest  
    json.NewDeCoder(r.Body).Decode(&req)  
  
    // 2. Map DTO to the Domain model  
    domainUser := mapper.MapCreateRequestToDomain(&req)  
  
    // 3. Use the repository INTERFACE to persist the user  
    err := s.userRepo.Create(domainUser)  
    // ... error handling ...  
  
    // 4. Present the result  
    data, _ := s.presenter.PresentUser(domainUser)  
    w.Write(data)  
}
```

Notice it calls the interface, not the concrete MySQL implementation.

Layer 3: The Infrastructure – Implementing the Details



- ****Purpose****: Provides concrete implementations for the interfaces defined in the domain layer (e.g., database logic, external API clients).
- ****Dependencies****: Implements interfaces from the Domain. The dependency is inverted.

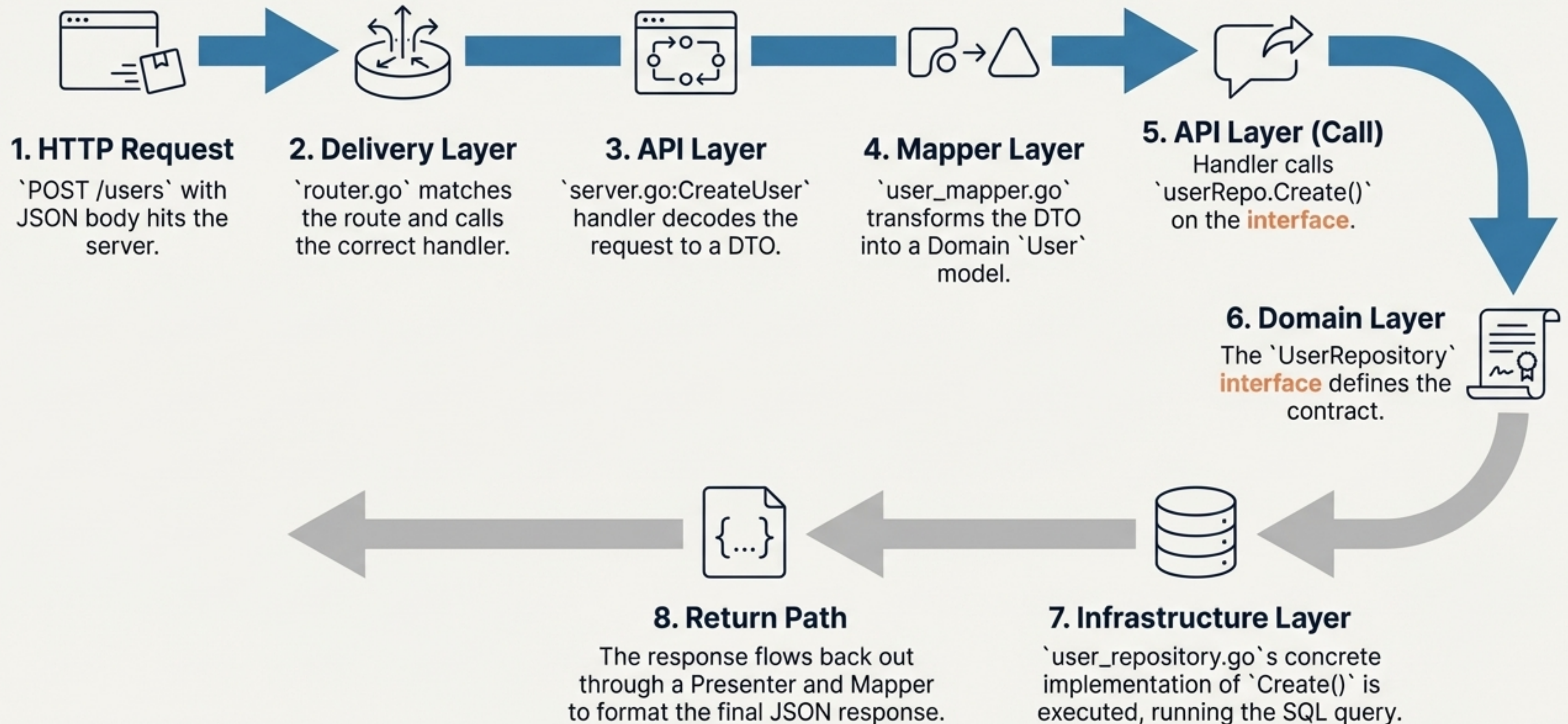
```
// Concrete implementation of the domain interface
type MySQLUserRepository struct {
    db *sql.DB
}

// Fulfills the UserRepository contract
func (r *MySQLUserRepository) Create(user *domain.User)
error {
    query := "INSERT INTO users (name, email, created_at,
        updated_at) VALUES (?, ?, ?, ?)"
    result, err := r.db.Exec(query, user.Name, user.Email,
        user.CreatedAt, user.UpdatedAt)

    // ...
    return nil
}
```

This is where the 'dirty' details like SQL live, completely isolated from the business rules.

Anatomy of a `POST /users` Request



A Modern, Containerized Go Stack



Docker & Docker Compose

For fully reproducible,
one-command development
environments.



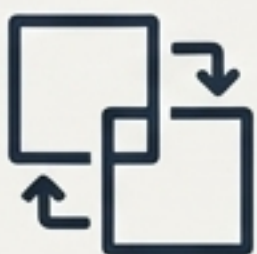
OpenAPI 3.0

For specification-first API design
and clear documentation
(`oapi.yaml`).



oapi-codegen

For generating Go server
boilerplate directly from the
OpenAPI specification.



jinzhu/copier

For reducing boilerplate and
errors in object mapping
between layers.



MySQL

The database implementation,
isolated in the infrastructure
layer.

The Result: A Maintainable and Testable System



Independent of Frameworks: Business rules don't depend on external libraries. You can swap the web server without touching the domain.



Highly Testable: Core business logic can be tested in complete isolation, without a UI, database, or external services.



Independent of Database: The business rules are not bound to MySQL. You can swap to Postgres by writing a new repository implementation.



Clear Separation of Concerns: Each layer handles specific responsibilities, making the codebase predictable and easier to navigate.

Explore the Code in 2 Commands

1. Clone the repository

```
git clone https://github.com/kumokuenchang/golang-clean-architecture.git
```

2. Run with Docker

```
docker-compose up --build
```

- This command builds and starts both the Go API and the MySQL database.
- The API will be available at: <http://localhost:8080>

Interacting with the API

Method	Endpoint	Description
POST	/users	Create a new user
GET	/users	Get all users
GET	/user?id={id}	Get a user by ID
PUT	/user?id={id}	Update user by ID
DELETE	/user?id={id}	Delete user by ID

Practical Example

```
# Create a new user
curl -X POST http://localhost:8080/users \
  -H "Content-Type: application/json" \
  -d '{"name": "John Doe", "email": "john@example.com"}'
```

Core Principles to Remember

Dependency Inversion

High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces).

Single Responsibility

Each component, from a function to a layer, has one primary reason to change.

Specification-First Development

Define your API contract in OpenAPI before writing the implementation code.



github.com/kumokuenchuan/golang-clean-architecture